Case 02 · Spec: Fire-Sale Cascade Simulator

Chapter 2 (Specification) of the Magshimim project-book template: features, technologies, flow diagram and database structure. The document is written so that another developer could build the system from it. Each feature is described at the level of "code in words": what it does, how it works step by step, which functions and data structures, which industry tools complement it, and what its inputs and outputs are.

System overview

What the system does, in one sentence

Fire-Sale Cascade Simulator builds a simulated financial market out of agents with real balance-sheet structures (leveraged pension funds, banks, credit funds, insurers), injects a shock into it, and shows round by round who is forced to sell, how their selling moves prices, and who falls next.

What changed since the POC and the threat model

- The POC worked with one agent type, one asset and one response rule. The full system must reproduce both the LDI crisis (leverage and collateral calls) and SVB (a deposit run on a bank with unrealised losses). These are two different mechanisms, so several agent types and several response rules are needed.
- The POC revealed that the outcome jumps from "safe" to "collapse" within a narrow range of one parameter. So the system never shows a single number: every result comes out as a range, with a sensitivity analysis showing which parameter drives the uncertainty (feature 8).
- **The threat model established two things:** the main threat is "engineering" a frightening result, and real data fed into the tool turn it into a sensitive target. So all computation runs on the user's machine only (feature 10), and "official" worlds are signed (feature 1).

Feature list

- Feature 1, World Builder & Agent Library
- Feature 2, Balance Sheet & Valuation
- Feature 3, Behavioral Rules Engine
- Feature 4, Market Impact Model
- Feature 5, Cascade Engine
- Feature 6, Exposure Network & Contagion
- Feature 7, Intervention Module
- Feature 8, Uncertainty Sweep & Sensitivity
- Feature 9, Historical Replay & Calibration
- Feature 10, Cascade View & Reproducible Links

Feature 1: World Builder & Agent Library

What does the feature do?

Creates the "world" the simulation runs on: a list of agents, each with a type, a balance sheet and constraints. A world can be built in three ways: from a ready world in the library (such as "Britain,

September 2022"), from distributions the user defines ("60 pension funds, average leverage between 3 and 5"), or from a table the user loads themselves.

How does it work?

Agent types

Each agent type is defined in a YAML file describing its balance-sheet structure and its response rules (feature 3). Five built-in types:

- **LDI Pension Fund:** holds long bonds to hedge pension liabilities, and levers itself through repo and interest-rate derivatives. When yields rise, the derivatives lose and collateral is called.
- **Commercial Bank:** holds bonds in two books, HTM (held to maturity, not marked in its accounts) and AFS (available for sale, marked), funded by deposits, some insured and some not.
- **Private Credit Fund:** lends at floating rates, funded by equity and credit lines. Exposed to borrower defaults when rates rise.
- **Insurer:** holds long bonds, with long liabilities and low leverage. Usually acts as a "buyer" rather than a seller.
- **Money Market Fund:** holds short bonds, exposed to investor redemption requests.

Three ways to build a world

From a template: choose a ready world from the library, which also includes the sources it is based on (central-bank financial-stability reports, public quarterly reports). From distributions: each parameter (equity, leverage, collateral thresholds, share of uninsured deposits) gets a distribution (truncated normal, log-normal or uniform), and the world is sampled with a fixed seed. From a table: a CSV with a column for every balance-sheet field. The table is checked against a schema and is never sent to a server (feature 10).

Signed worlds

In answer to the "spoofing a calibration" threat: every world in the official library is stored as a canonical JSON file signed with the project's Ed25519 key. When a world is loaded its signature is checked, and the interface shows a badge: "official world, version X" or "custom world". It is impossible to claim that a result is based on an official world when it is not.

Complementary industry technologies and infrastructure

- **Pydantic and JSON Schema:** every agent type and every world is defined as a Pydantic class that generates a JSON Schema automatically. Contribution: the table the user loads is checked against the same schema in the browser before it reaches the engine.
- **NumPy Generator (PCG64):** a modern seeded random number generator. Contribution: the same seed always produces exactly the same world on any machine, which is the basis for reproducibility (feature 10).
- **Mesa (as structural inspiration):** a leading Python ABM library that defines Agent and Model as classes. We adopt its conceptual structure, but not its agent-by-agent implementation, for performance reasons (feature 5).

Data structure

```
class AgentType(BaseModel):
    type_id: Literal["ldi", "bank", "pcf", "insurer", "mmf"]
    balance_fields: list[str]        # which BS lines exist for this type
    rule_set_id: str                 # -> RuleSet (feature 3)

class WorldSpec(BaseModel):
    world_id: str                    # sha256 of canonical spec
    name: str
    seed: int
    populations: list[Population]    # type_id, count, distributions
    assets: list[AssetSpec]          # gilts_long, gilts_short, corp_bonds, loans
    sources: list[str]               # where each preset number came from
    official: bool
    signature: bytes | None          # Ed25519, official worlds only
```

Input and output

Input: a world from the library, a set of distributions, or a CSV file. Output: a validated WorldSpec, and the balance-sheet arrays of every agent (feature 2).

Key libraries and functions

Operation	Library	Usage
Schema definition	Pydantic	BaseModel, model_json_schema()
Sampling	NumPy	default_rng(seed), lognormal(), truncated normal
Signature check	PyNaCl / tweetnacl-js	VerifyKey.verify() on the server, nacl.sign.detached.verify in the browser

Feature 2: Balance Sheet & Valuation

What does the feature do?

Holds every agent's balance sheet and revalues it whenever prices move. Without accurate valuation the question "who crossed the threshold" is meaningless: the difference between leverage of 4.9 and 5.1 is the difference between calm and a spiral, as the POC showed.

How does it work?

Balance-sheet lines

Each agent holds a vector of holdings (units of each asset) and liability lines: repo (collateralised borrowing), insured deposits, uninsured deposits, pension liabilities, equity. A "derivatives position" is also kept as a point sensitivity (DV01, how much the derivative gains or loses per basis point of yield), because that is what generates the LDI funds' collateral calls.

Bond valuation

The POC used a duration approximation only. The system uses a second-order approximation: the percentage price change equals minus duration times the yield change, plus half the convexity times the yield change squared. This approximation is accurate for moves of a few

percentage points, and it is vectorised. For checking, full valuation can be switched on: discounting every bond cash flow on an NSS curve (the same curve as in the debt simulator) and comparing with the approximation.

HTM versus AFS: the lesson of SVB

Bonds in the HTM book are not marked in the accounts, so equity "on paper" does not fall when yields rise. But if the bank is forced to sell them, the loss is realised all at once. The model keeps two equity numbers for every bank: accounting equity (excluding HTM losses) and economic equity (including them). Depositors' response rules (feature 3) react to economic equity, because that is what the market sees. That is exactly what happened to SVB in 2023.

Collateral haircuts

When an agent posts an asset as collateral for repo, the lender does not count its full value but applies a "haircut", for example 2% for a short government bond and 5% for a long one. Under stress haircuts rise, another mechanism that generates collateral calls even without a price fall. Haircuts are defined per asset as a function of recent volatility.

Complementary industry technologies and infrastructure

- **NumPy (Structure of Arrays):** a holdings matrix of agents times assets, a price vector, and a single matrix-vector product gives the asset value of every agent. Contribution: revaluing a thousand agents in microseconds.
- **QuantLib (for checking only):** full bond valuation on a curve, to verify that the second-order approximation is accurate enough over the range of moves the system simulates.
- **Basel III concepts (LCR, Leverage Ratio):** banks' regulatory liquidity and leverage ratios. Contribution: banks' response rules (feature 3) are written in terms a supervisory analyst knows, not terms we invented.

Data structure

```
class Balances:                  # Structure of Arrays, N agents x K assets
    holdings: np.ndarray         # (N, K) units held
    is_htm: np.ndarray           # (N, K) bool, banks only
    repo: np.ndarray             # (N,) secured funding
    dep_insured: np.ndarray      # (N,)
    dep_uninsured: np.ndarray    # (N,)
    pension_liab: np.ndarray     # (N,) PV of liabilities
    dv01_deriv: np.ndarray       # (N,) derivative P&L per bp
    cash: np.ndarray             # (N,)

class AssetState:                # K assets
    price: np.ndarray            # (K,)
    duration: np.ndarray         # (K,)
    convexity: np.ndarray        # (K,)
    haircut: np.ndarray          # (K,)
```

Input and output

Input: a WorldSpec and current asset prices. Output: for each agent, asset value, accounting equity, economic equity, leverage, and collateral gap.

Key libraries and functions

Operation	Library	Usage
Vectorised valuation	NumPy	holdings @ price, np.maximum
Valuation check	QuantLib	FixedRateBond, DiscountingBondEngine

Feature 3: Behavioral Rules Engine

What does the feature do?

Decides what each agent does in each round, according to its state. Rules are written as data (YAML), not code, so they can be read, changed and compared. This is critical because, as the research noted, simplistic rules are one of the model's biggest limitations, and they must be transparent.

How does it work?

Rule structure

Each rule has a condition, an action and a priority. The condition is an expression over balance-sheet fields (for example leverage > lmax, or collateral_gap > cash). The action is one of a closed list: SELL_TO_TARGET (sell until leverage returns to target), POST_COLLATERAL (post cash or assets), WITHDRAW (depositors withdraw), REDEEM (investors request redemption), DEFAULT. The priority decides what happens when several rules hold at once.

The rules for each type

- **LDI:** when the derivatives' DV01 produces a loss, collateral is called. If there is cash, it is used. If not, long bonds are sold. If equity is wiped out, DEFAULT.
- **Bank:** uninsured depositors withdraw according to a function of economic equity: the withdrawal rate in a round equals a logistic function of (threshold minus economic equity ratio), with a "sensitivity" parameter representing how fast the rumour spreads. When withdrawals exceed cash, the bank sells AFS and then HTM, realising the loss.
- **Private Credit Fund:** when rates rise, some borrowers fail to pay, according to a simulated interest coverage ratio (ICR). Losses reduce equity, and investors request redemption.
- **Insurer:** buys long bonds when the yield crosses a threshold, up to its risk appetite. This is the market's natural "breakwater".
- **Money Market Fund:** investors request redemption at a fixed rate or according to the fall in value, and the fund sells short bonds.

Selling order (liquidity waterfall)

When an agent needs cash, it sells in a defined order: the most liquid asset first, then the next. The order matters: it decides which market the selling hits, and therefore who gets hurt. A bank selling short bonds first hurts money market funds. A pension fund selling long bonds hurts other pension funds.

Complementary industry technologies and infrastructure

- **YAML with JSON Schema:** rules are stored as readable text and checked against a schema. Contribution: an analyst can read a rule without reading Python.
- numexpr (on the server) or a simple expression compiler of our own (in the browser): converts an expression such as "leverage > lmax" into one vector operation over all agents at once. Contribution: rules as data, without paying for it in performance. In the browser, a small expression library translates to NumPy operations, with no eval on user input.
- **The Diamond and Dybvig (1983) bank-run model:** the classic model of a run on a bank, for which its authors received the Nobel prize in economics in 2022. Contribution: our withdrawal rule is a simple, dynamic version of its central idea: each depositor withdraws because they fear others will.

Data structure

```
rule_set: ldi_v1
rules:
  - id: LDI_MARGIN
    when: "collateral_gap > 0"
    then: POST_COLLATERAL
    priority: 1
  - id: LDI_FIRE_SALE
    when: "collateral_gap > cash"
    then: {action: SELL_TO_TARGET, order: [gilts_short, gilts_long]}
    priority: 2
  - id: LDI_DEFAULT
    when: "equity_economic <= 0"
    then: DEFAULT
    priority: 0
```

Input and output

Input: balance-sheet state after valuation, and a RuleSet per type. Output: for each agent, the chosen action and the quantity sold in each asset.

Key libraries and functions

Operation	Library	Usage
Read rules	PyYAML, jsonschema	safe_load(), validate()
Vector evaluation	numexpr	evaluate("leverage > lmax", local_dict)
Run function	NumPy	1 / (1 + exp(-k * (thr - ratio)))

Feature 4: Market Impact Model

What does the feature do?

Translates "how much was sold" into "how far the price moved". This is where one agent's selling becomes another agent's problem, so it is the most sensitive part of the model. The POC used a single linear function. The system supports several models, and shows the user how much the result depends on the choice between them.

How does it work?

Three models

- **Linear (Kyle's lambda):** the price change is proportional to the quantity sold. Simple, and suitable for small moves. Named after Albert Kyle's 1985 model.
- **Square-root law:** the price change is proportional to the square root of the quantity relative to daily trading volume. This is the best-known empirical law in the field, measured again and again across markets. It means that selling twice as much does not move the price twice as far, but about 1.4 times as far.
- **With dealer capacity:** dealers (banks that make markets) have limited balance sheets. Up to a limit they absorb selling with moderate impact; beyond it the impact jumps. This reflects what happened in Britain in 2022: the market worked, until suddenly it did not.

Cross-impact

Selling long bonds also moves medium-term bonds, because investors switch between them. A cross-impact matrix of assets times assets determines by how much. The default is derived from the historical correlation between bond yields.

Recovery

After a round of selling, part of the impact "recovers" in the next round as new buyers come in (insurers, for example, feature 3). The recovery rate is a parameter: a price that does not recover at all produces a spiral faster.

Complementary industry technologies and infrastructure

- **The Almgren and Chriss market-impact models:** the industry's standard framework for separating temporary from permanent impact, used by every algorithmic trading desk. Contribution: our separation between impact and recovery rests on a familiar framework.
- Cont and Schaanning, "Fire Sales, Indirect Contagion and Systemic Stress Testing": work that defines contagion through "common holdings": two agents that owe each other nothing but hold the same asset hurt each other through the price. Contribution: our model explicitly captures this channel.

Data structure

```
ImpactModel {
  kind: Literal["linear", "sqrt", "dealer_capacity"]
  lam: np.ndarray            # (K,) per-asset impact coefficient
  adv: np.ndarray            # (K,) average daily volume, for sqrt law
  dealer_cap: np.ndarray     # (K,) dealer absorption capacity
  cross: np.ndarray          # (K, K) cross-impact matrix, diag = 1
  recovery: float            # share of impact that decays per round
}
```

Input and output

Input: the vector of net sales in each asset in the round, and asset state. Output: a vector of price changes, and the corresponding yield change for each bond.

Key libraries and functions

Operation	Library	Usage
Vector impact	NumPy	sign(q) * lam * sqrt(abs(q) / adv)
Cross-impact	NumPy	cross @ own_impact

Feature 5: Cascade Engine

What does the feature do?

This is the technological core. The engine runs the rounds: valuation, decisions, sales, price impact, and repeat, until the market calms or the maximum number of rounds is reached. The output is the full history of every round.

How does it work?

The round loop

- **Valuation:** all balance sheets are revalued at current prices (feature 2).
- **Decision:** the rules engine decides what each agent does (feature 3).
- **Settlement:** sales are converted to cash at the current price less temporary impact, cash pays repo and withdrawals, and agents that reached DEFAULT dump all their assets on the market.
- **Intervention:** if defined, the central bank or another mechanism absorbs part of the selling (feature 7).
- **Price impact:** the remaining net sales move prices (feature 4).
- **Direct contagion:** an agent's default cuts the value of loans others made to it (feature 6).
- **Recording:** every state is saved as a RoundState.

Stopping conditions

The engine stops when forced sales in a round fall below a threshold (0.01% of total market assets), when the price change in a round is negligible, or after 100 rounds. If it stopped because of the cap rather than convergence, the result is marked "did not converge", because that signals the model is in a spiral that does not end.

Conservation checks

Every round: total cash in the system, plus asset value, minus recorded losses, must reconcile with the previous round. Units sold must equal units bought (by the central bank, the insurers, or the abstract "market" behind the impact model). A violation stops the run with an error.

Rounds and the real clock

A round is not a day. In the LDI crisis the spiral unfolded over several days, and in the case of SVB within a day and a half. Every world in the library has an "estimated round length", so the view can say "about two days" and not just "6 rounds", with an explicit caveat that this is an estimate.

Complementary industry technologies and infrastructure

- **NumPy (full vectorisation):** the POC measured 0.28 seconds for 1,000 runs of 60 agents. The full system, with 5 types and 4 assets, is expected to be several times slower, and still under a second for a sweep of hundreds of worlds.
- **Numba (on the server, for calibration):** calibration (feature 9) needs tens of thousands of runs. Compiling the round loop speeds it up tens of times.
- **Pyodide with a Web Worker:** runs in the browser, in the background, without blocking the interface.

Data structure

```
RoundState {
  round: int
  prices: np.ndarray            # (K,)
  yields_bp: np.ndarray         # (K,) change vs start
  forced_sales: np.ndarray      # (N, K)
  cb_absorbed: np.ndarray       # (K,)
  equity_econ: np.ndarray       # (N,)
  leverage: np.ndarray          # (N,)
  status: np.ndarray            # (N,) int8: 0 ok, 1 acting, 2 default
}

CascadeResult {
  run_id: str
  rounds: list[RoundState]
  converged: bool
  amplification: float          # final yield move / initial shock
  share_forced: float
  defaults: int
}
```

Input and output

Input: a world, a shock (an initial yield change, or an initial deposit withdrawal), and an optional intervention. Output: a CascadeResult.

Key libraries and functions

Operation	Library	Usage
Vector computation	NumPy	masking, np.add.reduce
Server acceleration	Numba	@njit(cache=True)
Browser execution	Pyodide	runPythonAsync inside a Worker

Feature 6: Exposure Network & Contagion

What does the feature do?

There are two channels through which one agent brings down another. The first is through prices (feature 4): I sold, the price fell, and you lose. The second is direct: I lent to you, you failed, and I lose the loan. This feature models the second channel, and identifies which agents are "central nodes" whose fall brings down the most.

How does it work?

The network

A directed graph in which every edge is a loan: who lent to whom and how much (repo, interbank loans, credit lines to funds). In worlds built from distributions, the graph is generated randomly using a "core-periphery" model: a few large banks at the core lending to each other and to everyone, and many small agents on the periphery. This is the structure actually measured in interbank networks.

Contagion

When an agent reaches DEFAULT, everyone who lent to it takes a loss equal to the loan times (1 minus the recovery rate). The recovery rate is a parameter (default 40%). The loss reduces equity, and can push the next agent over a threshold, start it selling, or bring it down too.

DebtRank

To identify central nodes, the feature computes DebtRank for every agent: an algorithm from 2012 (Battiston and colleagues) that measures what share of the economic value in the network would be hit if this agent lost its equity. The algorithm resembles Google's PageRank, but with a mechanism that prevents counting the same loss twice around cycles in the network. The result: a ranking of who is "systemically dangerous" inside the simulated world.

Complementary industry technologies and infrastructure

- **NetworkX:** Python's standard graph library. Contribution: building the network, computing centrality measures for comparison, and exporting for display.
- **SciPy sparse:** the network is sparse (most agents do not lend to most agents), so it is stored as a sparse matrix. Contribution: computing contagion is a sparse matrix-vector product, fast even for thousands of agents.
- **DebtRank (Battiston et al., Scientific Reports, 2012):** the systemic-risk measure the ECB and other central banks have examined in research. Contribution: "who is most dangerous" is measured by a known algorithm, not by feel.

Data structure

```
ExposureGraph {
  lender: np.ndarray        # (E,) agent index
  borrower: np.ndarray      # (E,)
  amount: np.ndarray        # (E,)
  kind: np.ndarray          # (E,) int8: repo, interbank, credit_line
  recovery: float
}
DebtRankResult { agent_index: int; debtrank: float; rank: int }
```

Input and output

Input: a list of loans (or a core-periphery definition), and the list of agents that fell in the round. Output: contagion losses for each agent, and DebtRank for each agent.

Key libraries and functions

Operation	Library	Usage
Graph	NetworkX	DiGraph, from_scipy_sparse_array
Contagion	SciPy	sparse.csr_matrix, dot
Centrality	our own implementation	DebtRank as in the original paper

Feature 7: Intervention Module

What does the feature do?

The POC showed that an intervention does not prevent selling; it breaks the loop. This feature makes it possible to test different kinds of intervention, when they are triggered and at what size, and to compare them on the same world.

How does it work?

Four tools

- **Central-bank purchases:** the central bank buys a given type of bond, up to a daily cap and a total cap, for a limited period. Inspired by the Bank of England's intervention in September 2022: it announced temporary purchases of long gilts of up to £5bn a day for 13 business days, and in practice bought only about £19bn. The announcement itself calmed the market.
- **Collateral relief:** cutting haircuts or extending the deadline for collateral calls.
- **Extended deposit insurance:** all deposits are insured, so depositors do not withdraw. Inspired by the decision on SVB, March 2023.
- **A par-value lending facility:** banks can borrow from the central bank against bonds at their face value rather than their market price, and so are not forced to sell at a loss. Inspired by the Bank Term Funding Program (BTFP) the Fed set up in March 2023.

Triggers

Every intervention fires on a trigger: from a given round, or when an indicator crosses a threshold (for example a yield move of more than 100 basis points). This makes it possible to test a question such as "what if the central bank had responded one round earlier".

The effect of the announcement

In reality, part of an intervention's effect comes from the announcement itself, before a single purchase. The feature models this as a "credibility" parameter: a credible announcement lowers the impact coefficient (feature 4) from the round in which it is announced. The parameter is clearly marked as an assumption.

Complementary industry technologies and infrastructure

- **Official documentation of the interventions:** the Bank of England's announcements from September 2022, and the Fed's BTFP terms. Contribution: every tool in the feature is based on an intervention that really happened, with documented terms.
- **The Strategy pattern:** a design pattern in which every intervention is a class with the same interface (apply(state) returns a state). Contribution: a new kind of intervention can be added without touching the engine.

Data structure

```
class Intervention(Protocol):
    def triggered(self, state: RoundState) -> bool: ...
    def apply(self, state: RoundState, sales: np.ndarray) -> np.ndarray: ...

CBPurchase { asset: int; daily_cap: float; total_cap: float; rounds: int; credibility:
float }
CollateralRelief { haircut_cut: float; grace_rounds: int }
DepositGuarantee { coverage: Literal["insured_only", "all"] }
ParValueFacility { eligible_assets: list[int]; rate_spread_bp: float }
```

Input and output

Input: a list of interventions with triggers. Output: net sales after absorption, and cost measures for each intervention (how much was bought, how much was lent).

Key libraries and functions

Operation	Library	Usage
Uniform interface	typing.Protocol	Intervention
Comparing interventions	concurrent.futures	parallel runs on the same world

Feature 8: Uncertainty Sweep & Sensitivity

What does the feature do?

This is the feature the POC made mandatory. Instead of "the yield will jump 1.13 points", the system says "between 0.8 and 1.6, and 70% of that uncertainty comes from fund leverage, which we do not know". It is also the main answer to the "engineering the result" threat: a frightening result is always shown together with its range and the parameter responsible for it.

How does it work?

Sweep

The user marks which parameters are unknown and gives each a range. The feature generates N worlds (default 512) by Sobol sampling, which covers the parameter space more evenly than random sampling. Each world is run through the cascade engine. Output: a distribution of every outcome measure, with percentiles 10, 50 and 90.

Global sensitivity analysis

Sobol indices decompose the variance of the outcome by parameter: S1 is the share each parameter explains alone, and ST is the share it explains including interactions with others. The display is a "tornado chart": parameters sorted by ST. If leverage has an ST of 0.7, then 70% of the uncertainty in the outcome depends on it.

Finding the tipping point

The POC found a threshold: between leverage of 4 and 5.5 the outcome jumps. The feature finds this threshold automatically: for each parameter, a binary search for the value at which the

probability of a spiral (a move above a defined threshold) crosses 50%. The result is shown as a sentence: "above average leverage of 4.7, a spiral is more likely than not".

A default that cannot be bypassed

A single-world result is always shown labelled "single run, see range", with a link to a sweep of the same world over the default ranges. It is impossible to download an image of a single run without that label in it.

Complementary industry technologies and infrastructure

- **SALib:** the most widely used open Python library for sensitivity analysis, implementing Saltelli sampling and Sobol indices. Contribution: sensitivity indices computed by the method accepted in research, not one we invented.
- **scipy.stats.qmc.Sobol:** quasi-random sampling. Contribution: a stable fan with fewer runs.
- **Uncertainty Quantification (UQ) as a field:** the engineering discipline devoted to exactly this, in climate, aviation and nuclear simulation. Contribution: we are not inventing the approach; we are applying an established field to a financial problem.

Data structure

```
SweepSpec { world_id; params: list[ParamRange]; n: int; seed: int; spiral_threshold_bp:
float }
ParamRange { name: str; low: float; high: float; dist: Literal["uniform", "loguniform"] }
SweepResult {
  sweep_id: str
  outcome_quantiles: dict[str, tuple[float, float, float]]   # p10, p50, p90
  p_spiral: float
  sobol: dict[str, tuple[float, float]]                      # S1, ST
  tipping_points: dict[str, float | None]
}
```

Input and output

Input: a world and a list of parameters with ranges. Output: a SweepResult with ranges, spiral probability, sensitivity indices and tipping points.

Key libraries and functions

Operation	Library	Usage
Sampling	SALib	sample.saltelli / sample.sobol
Sensitivity	SALib	analyze.sobol
Tipping point	SciPy	optimize.bisect on p_spiral(x) - 0.5

Feature 9: Historical Replay & Calibration

What does the feature do?

For anyone to trust the simulation, it must show that it behaves like reality in cases we know. This feature calibrates the parameters so that the model reproduces two real crises, and shows openly how well (or badly) it does.

How does it work?

The two test cases

- **Britain, September 2022:** target data from the Bank of England's public reports: the size of the move in long gilt yields in the days after the budget, the actual scale of purchases, and the recovery after the announcement.
- **SVB, March 2023:** target data from the bank's and the Fed's public reports: the share of uninsured deposits (the great majority, about 90%), the unrealised losses in the HTM book, and the scale of withdrawals on the day of collapse (about $42bn in a single day).

Calibration method

The model gives no closed-form distribution of outcomes, so ordinary statistical methods cannot be used. The feature uses Approximate Bayesian Computation (ABC): sample parameters from a prior, run, and keep only the samples whose outcome is close enough to the target data. What remains is a distribution of parameters that "explain" the crisis. In the advanced version (ABC-SMC) the tolerance shrinks gradually, which reduces the number of runs.

Fit report

For each test case a report is shown: for every target, the true value against the range of values the calibrated model produces. A target the model fails to reproduce is marked in red, not hidden. For example: if the model reproduces the size of the move but not the speed of recovery, that is stated.

Cross-validation

Calibration is done on one case and tested on the other, to the extent the mechanisms overlap (price-impact parameters, for example). If parameters calibrated on Britain also produce a plausible result under SVB conditions, there is more reason to trust them.

Complementary industry technologies and infrastructure

- **pyabc:** an open Python library for ABC-SMC with parallel execution. Contribution: calibration by an accepted method, with a parameter distribution rather than "one value that works".
- **Simulated Method of Moments (as an alternative):** a method common in economics for calibrating agent models, matching moments (mean, variance) of the simulation to the data. Contribution: a cross-check on the ABC results.
- **Dask (for calibration on the server):** a library for parallel computation across processors or machines. Contribution: the tens of thousands of calibration runs execute in parallel.

Data structure

```
CalibrationTarget { case_id: Literal["uk_ldi_2022", "svb_2023"]; metric: str; value:
float; tolerance: float; source: str }
CalibrationRun { calib_id; case_id; method: Literal["abc_smc", "smm"]; n_sims: int;
accepted: int; posterior_path: str }
FitReport { calib_id; metric; target; sim_p10; sim_p50; sim_p90; within_tolerance: bool }
```

Input and output

Input: target data from public sources, and prior distributions for the parameters. Output: a calibrated parameter distribution, and a public fit report.

Key libraries and functions

Operation	Library	Usage
Calibration	pyabc	ABCSMC(models, priors, distance)
Parallel execution	Dask	distributed.Client
Posterior storage	Parquet	a posterior per calibration

Feature 10: Cascade View & Reproducible Links

What does the feature do?

The interface: an animation of the cascade round by round, a network map, a sensitivity tornado chart, and a link that reproduces any run exactly. And most important of all: all computation runs in the user's browser, and no balance-sheet data ever leave their computer.

How does it work?

The cascade animation

Each agent is a dot, coloured by state (calm, acting, failed) and sized by balance-sheet total. In each round the selling dots "emit" particles towards the market, the yield gauge moves, and dots that crossed a threshold change colour. Below the animation: a chart of the yield across rounds, and a live caption describing the round in words ("Round 3: 12 funds received collateral calls, 4 sold long gilts"). The caption is accessible to screen readers.

Local execution only

In answer to the "leaking real balance sheets" threat: the engine runs on Pyodide inside a Web Worker. A table the user loads is read with the FileReader API directly into browser memory, and no network request carries it. Run history is kept in local IndexedDB, and the user can delete it with one click. The server holds only static files: the code, the signed library and the calibrations.

Reproducible link

The link contains the world_id (of an official world or a set of distributions), the seed, the shock, the interventions and the engine version, encoded in the URL fragment (after #), which the browser does not send to the server. In answer to the "result without history" threat: whoever opens the link runs exactly the same computation. For a world built from a private table no link is created, and the interface explains why.

Resource limits

A sweep is capped at 4,096 worlds in the browser, with a time estimate before running and the option to cancel midway. This answers the "resource exhaustion" threat.

Complementary industry technologies and infrastructure

- **D3.js with d3-force:** a force simulation for laying out the network, so connected agents move closer together. Contribution: a readable network map without manual layout.
- **Canvas 2D:** for animating hundreds of dots and particles, where SVG would be too slow.
- **IndexedDB:** a database inside the browser. Contribution: run history that stays with the user only.

- **Content Security Policy:** an HTTP header that restricts where the page may send requests. Contribution: even a bug or a malicious library cannot send balance-sheet data out, because the browser blocks any request not addressed to our own server.

Data structure

```
RunLink {           # encoded in URL fragment, never sent to server
  world_id: str
  seed: int
  shock: { kind: "yield" | "deposit_run"; size: float }
  interventions: list[dict]
  engine_version: str
}
LocalRun { run_id; created_at; link: RunLink | None; result: CascadeResult; private_world:
bool }
```

Input and output

Input: CascadeResult, SweepResult, ExposureGraph. Output: an animation, a network map, a tornado chart, a reproducible link, and a downloadable image that always includes the labels and assumptions.

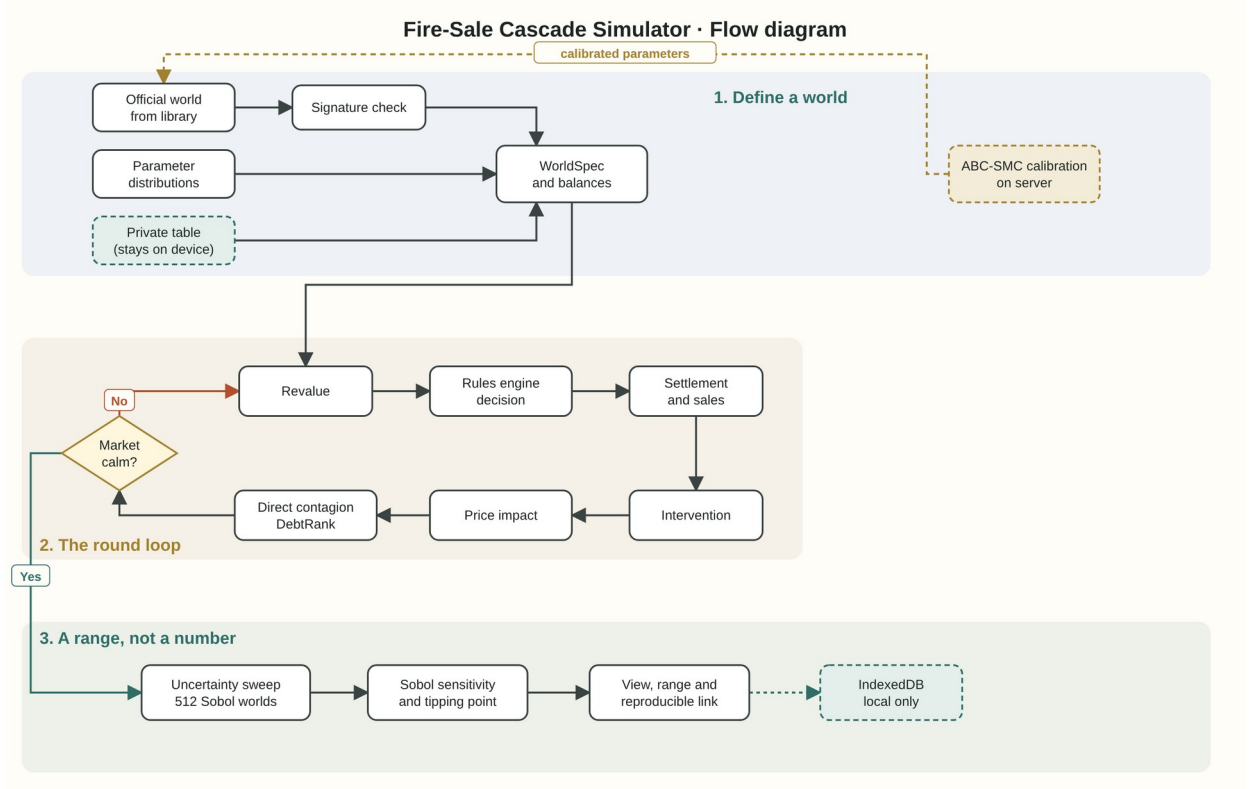
Key libraries and functions

Operation	Library	Usage
Network layout	d3-force	forceSimulation, forceLink
Animation	Canvas 2D	requestAnimationFrame
Local storage	IndexedDB	idb-keyval
Leak prevention	CSP	connect-src 'self'

Technology table

Feature	Technologies and languages	External resources and services
1. World building	Python, Pydantic, NumPy, PyNaCl	Signed world library (static files)
2. Balance sheet and valuation	NumPy, QuantLib	None
3. Response rules	YAML, jsonschema, numexpr	None
4. Price impact	NumPy	Public trading-volume data for defaults
5. Cascade engine	NumPy, Numba, Pyodide	None (runs in the browser)
6. Network and contagion	NetworkX, SciPy sparse	None
7. Intervention	Python (Protocol)	Official BoE and Fed documentation
8. Sweep and sensitivity	SALib, SciPy qmc	None
9. Replay and calibration	pyabc, Dask, Parquet	Public BoE, Fed and SVB reports
10. View and link	D3.js, Canvas, IndexedDB, CSP	CDN for static files only

Flow diagram



The diagram has three stages. In setup (top), the user builds a world in one of three ways: an official world whose signature is checked, distributions, or a private table that stays on their machine. In the engine (middle), the round loop: revalue, decide, settle, intervene, price impact and contagion, and back to revaluation, until the market calms. In the result (bottom), a single run never goes out on its own: it always passes through the uncertainty sweep, and only then is it shown with its range, sensitivity chart and tipping point. Calibration (on the right) runs separately on the server and feeds calibrated parameters into the official worlds.

Database structure

The system has two stores. Server side: PostgreSQL for the public library (agent types, rules, official worlds, calibrations), published as signed static files. User side: IndexedDB for run history, which never leaves the browser. There is no table on the server that contains a user's balance-sheet data.

Server side: table agent_types

- **type_id:** VARCHAR(16), primary key ('ldi', 'bank', 'pcf', 'insurer', 'mmf').
- **display_name:** VARCHAR, required.
- **balance_fields:** JSONB, required (the balance-sheet lines this type has).
- **default_rule_set:** foreign key to rule_sets, required.

Table rule_sets

- **rule_set_id:** VARCHAR, primary key (for example 'ldi_v1').
- **type_id:** foreign key to agent_types.

- **yaml_source:** TEXT, required.
- **sha256:** CHAR(64), required, unique.
- **created_at:** TIMESTAMPTZ.

Table worlds

- **world_id:** CHAR(64), primary key (SHA-256 of the canonical definition).
- **name:** VARCHAR, required.
- **spec:** JSONB, required (a full WorldSpec).
- **official:** BOOLEAN, default false.
- **round_length_hint:** VARCHAR (for example 'about half a day'), may be NULL.
- **calib_id:** foreign key to calibration_runs, may be NULL.

Table world_signatures

- **world_id:** foreign key to worlds, primary key.
- **key_id:** VARCHAR, required.
- **signature:** BYTEA (64 bytes), required.
- **signed_at:** TIMESTAMPTZ, required.

Table world_sources

- **world_id:** foreign key to worlds.
- **field_path:** VARCHAR (for example 'populations[0].leverage.mean'), required.
- **source_title:** VARCHAR, required.
- **source_url:** VARCHAR, required.
- **Composite primary key:** (world_id, field_path).

Table calibration_targets

- **target_id:** SERIAL, primary key.
- **case_id:** VARCHAR, required ('uk_ldi_2022', 'svb_2023').
- **metric:** VARCHAR, required.
- **value:** DOUBLE PRECISION, required.
- **tolerance:** DOUBLE PRECISION, required.
- **source_url:** VARCHAR, required.

Table calibration_runs

- **calib_id:** UUID, primary key.
- **case_id:** VARCHAR, required.
- **method:** VARCHAR, required ('abc_smc', 'smm').
- **n_sims:** INTEGER, required.
- **accepted:** INTEGER, required.
- **posterior_path:** VARCHAR, required (a Parquet file).
- **engine_version:** VARCHAR, required.

Table fit_reports

- **calib_id:** foreign key to calibration_runs.
- **metric:** VARCHAR.
- **target, sim_p10, sim_p50, sim_p90:** DOUBLE PRECISION.
- **within_tolerance:** BOOLEAN, required.
- **Composite primary key:** (calib_id, metric).

User side (IndexedDB): store runs

- **run_id:** key (SHA-256 of the RunLink, or a UUID for a private world).
- **created_at:** date.
- **link:** a RunLink object, or null for a private world.
- **private_world:** boolean.
- **result:** a compressed CascadeResult (rounds as typed arrays).

Store sweeps

- **sweep_id:** key.
- **run_id:** reference to the runs store.
- **spec:** SweepSpec.
- **result:** SweepResult.

Store private_worlds

- **world_local_id:** key (UUID).
- **name:** string.
- **balances:** typed arrays of the balance sheets.
- **Note:** deleted in full with one click, and never backed up on the server.

What the spec closed, and what remains open

- **Closed:** the full model covers both mechanisms from the story (leverage and collateral, a deposit run), both contagion channels (prices and direct debts), and four kinds of intervention that really happened. All six threats in the threat model are answered: signed worlds, a range and sensitivity with every result, a reproducible link, local-only execution, resource limits, and a statement of the tool's purpose.
- **Open:** the POC showed, and the spec does not change this, that the outcome depends mainly on exposure data we cannot see. Feature 8 can say exactly how much the answer depends on them, and feature 9 can calibrate on past crises. But about today's market, the tool still answers "it depends on leverage we do not know". It knows how to compute what will happen, if only we knew who holds what.